

Software Requirements Specification

CareSlot – Appointment Management System

Project Title	CareSlot – Appointment Management System		
Student Name	Riddhi Tanwar	Roll Number	22515000110
Course / Semester	B.Tech CSE – 3rd Semester	Subject	Frontend Engineering
Project Guide	Mr. Gautam Mukherjee	Institute	GLA University, Greater Noida Campus
Document Type	Software Requirements Specification (SRS)	Version / Date	1.0 / 9th October 2026

Revision History

Version	Date	Author	Description
1.0	9th October 2026	Riddhi Tanwar	Initial SRS prepared for the CareSlot project

Table of Contents

Section	Title	Section	Title
1	Introduction	8	Design Constraints and Coding Standards
2	Overall Description	9	Validation Rules
3	System Structure	10	Test Cases and Traceability
4	External Interface Requirements	11	Future Enhancements
5	Functional Requirements	12	Conclusion
6	Content and Field Definitions		
7	Non-Functional Requirements		

1. Introduction

1.1 Purpose

This document states what the CareSlot Appointment Management System shall do. It turns the project idea into specific, checkable requirements, fixes the scope before coding is finalised, gives the project guide a clear list of deliverables, and is the reference for viva and evaluation. Anything not written in this document is not part of the project.

1.2 Document Conventions

Requirements are numbered: FR-5.2 means Functional Requirement, module 5, point 2. **Shall** means compulsory; **may** means optional. Prefixes: FR Functional, NFR Non-Functional, CON Constraint, ASM Assumption, NAV Navigation, UI Interface, STD Coding Standard, VAL Validation, TC Test Case.

1.3 Intended Audience

Guide / Evaluator: Sections 1, 2, 5, 7, 10. Student / Developer: whole document, especially 3, 5, 6, 9. Viva examiner: Sections 2.7, 7, 9, 11.

1.4 Project Scope

CareSlot is a front-end-only, single-page web application for the reception desk of a small clinic. The user can book an appointment, see all appointments in one table, search and filter them, edit (reschedule), complete, cancel or delete them, and see live counts of total, upcoming, completed and cancelled appointments. It is built with HTML5, CSS3, JavaScript (ES6) and Bootstrap 5. There is no server, no database and no login; appointments are saved in the browser's `localStorage` and survive a refresh on the same device. This is a deliberate decision: the project demonstrates DOM handling, events, custom validation, CRUD operations, browser storage and responsive design. Multi-user access and central storage belong to a later phase (Section 11).

1.5 Definitions, Acronyms and Abbreviations

Term	Meaning	Term	Meaning
localStorage	Browser storage area that keeps text data on that device	CRUD	Create, Read, Update, Delete
JSON	Text format used to save the appointments array	DOM	The browser's live model of the page, changed by JavaScript
Bootstrap / CDN	CSS library, loaded from an internet server	Event delegation	One listener on a parent handles all child buttons
XSS	Attack where typed code gets run by the page	Double booking	Two active bookings, same doctor, date and time

1.6 References

IEEE Std 830-1998 (structure adapted); W3C HTML5 and CSS3 specifications; MDN Web Docs (Web Storage API, Date, Array methods); Bootstrap 5.3 documentation; WCAG 2.1.

2. Overall Description

2.1 Product Perspective

A new, self-contained product that does not replace or connect to existing software. Clinics now use paper registers and phone slips: entries are hard to search, slips get lost, double bookings are found only when patients arrive, and nobody counts cancelled or completed visits. CareSlot puts all of this on one screen. Opening `index.html` in a browser is enough to run it.

2.2 Product Functions

The system shall: (1) book appointments using patient name, phone, doctor, date, time and notes; (2) validate every entry and block past-time and double bookings; (3) list appointments sorted by time with status badges; (4) search by patient or doctor and filter by status; (5) edit, complete, cancel or delete; (6) show live statistics; (7) save everything in the browser; (8) work on phone and laptop screens.

2.3 User Classes and Characteristics

User class	Skill	How they use the site
Receptionist / front-desk staff (primary)	Basic	Books, edits, completes, cancels and deletes appointments
Clinic manager / doctor (secondary)	Basic	Reads the statistics; searches the list by doctor name
Evaluator / guide	High	Inspects the page, code and this document

There is no login, so all users are equal and there are no roles or permissions.

2.4 Operating Environment

Any desktop, laptop, tablet or phone (320 px and above) with a current Chrome, Firefox, Edge or Safari and JavaScript enabled; no server, no database. Internet is needed only to load Bootstrap and fonts.

2.5 Design and Implementation Constraints

ID	Constraint
CON-1	Only HTML5, CSS3, vanilla JavaScript (ES6) and Bootstrap 5.3.3 shall be used; no React, Angular, Vue or jQuery
CON-2	No backend, server-side language or database; data is stored only in <code>localStorage</code>
CON-3	One HTML page (<code>index.html</code>) with one stylesheet (<code>style.css</code>) and one script (<code>script.js</code>)
CON-4	No external API or online service shall be called
CON-5	Bootstrap and the Google Fonts DM Sans and Fraunces are loaded from a CDN; nothing else comes from the internet
CON-6	The site shall work when opened from the file system (<code>file://</code>) as well as from a server

2.6 Assumptions and Dependencies

ID	Assumption
ASM-1	The browser supports ES6, <code>localStorage</code> , CSS custom properties, Flexbox, Grid and date/time inputs; <code>localStorage</code> is not blocked (private windows erase it on close)
ASM-2	One person uses the app at a time on one device; the device clock is correct (the past-time check uses it)
ASM-3	Internet is available at first load; if Bootstrap fails, the layout degrades, and if fonts fail, fallback fonts are used
ASM-4	Doctor names, patient names and phone numbers used in demonstration are fictional; real patient data is not entered

2.7 What the System Deliberately Does Not Do

It does not: provide login or roles; share data between devices or users; send SMS, email or reminders; store medical history; know doctors' working hours or appointment length (it only blocks two active bookings at exactly the same time); auto-complete appointments whose time has passed; move a Completed or Cancelled appointment back to Upcoming; export or print; undo a delete; accept payments. Each appears in Section 11.

3. System Structure

3.1 Page Map and Folder Structure

```
index.html (the only page)
|-- Navigation bar: CareSlot | Book | Appointments
|-- Hero: headline + description
|-- Statistics: Total, Upcoming, Completed, Cancelled
|-- #book Booking form (also used for editing)
|-- #list Search + filter + table + row actions
`-- Footer

Layout: form (1/3) beside list (2/3) on large screens; stacked, form first, on small screens

careslot/
|-- index.html  structure (elements)
|-- style.css   looks (colours, fonts)
|-- script.js   behaviour (validate, save)
`-- README.txt  how to run
```

3.2 How the Files Connect

File	Job	Connects through
<code>index.html</code>	Structure: form, table, buttons, cards	Links <code>style.css</code> (after Bootstrap) and <code>script.js</code> (end of body, after Bootstrap JS)
<code>style.css</code>	Looks: colours, fonts, spacing, animation	Styles elements by class names such as <code>.panel</code> , <code>.stat</code> , <code>.badge-Upcoming</code>
<code>script.js</code>	Behaviour: validate, save, edit, search, draw	Finds elements by id (e.g. <code>getElementById('name')</code>); ids and class names must match exactly

3.3 Navigation Rules

NAV-1 The navbar has the brand and two links, Book (`#book`) and Appointments (`#list`). **NAV-2** Clicking the brand returns to the top. **NAV-3** The edit button scrolls smoothly to the form. **NAV-4** The page stays usable at 320 px without sideways page scrolling.

3.4 JavaScript Function Map

Function	What it does	Function	What it does
save()	Writes the array to localStorage as JSON	resetForm()	Returns the form to booking mode
escapeHTML()	Makes typed text safe (blocks XSS)	editAppointment(id)	Fills the form and enters edit mode
formatDate()	Shows '12 Oct 2026, 02:30 pm'	setStatus(id, s)	Marks Completed or Cancelled
showError() / clearError()	Shows or hides the red box and borders	deleteAppointment(id)	Confirms, then removes the record
validate(data)	Runs all checks; returns error or null	render()	Filters, sorts, redraws table and stats
Form submit handler	Builds record, validates, creates or updates	Table click handler	One delegated listener for all row buttons

4. External Interface Requirements

4.1 User Interface Requirements

ID	Requirement
UI-1	One colour scheme (deep teal, mint, amber accent, off-white page, dark teal-grey text), defined once in :root
UI-2	Two font families only: Fraunces for headings and brand, DM Sans for body, each with a fallback
UI-3	Base font 13 px on phones and 15 px at 992 px and above; other sizes in rem
UI-4	Visible hover change on clickable items; teal outline on focused form fields
UI-5	Every field has a visible label; placeholder text alone is not a label
UI-6	Errors appear in a red alert box inside the form and the faulty fields get a red border
UI-7	Status badge colours: blue Upcoming, green Completed, red Cancelled, each also carrying the status word
UI-8	A new or updated row flashes light amber, fading in about 1.2 seconds

4.2 Hardware, Software and Communication Interfaces

Hardware: none; only a display and a keyboard, mouse or touchscreen. **Software:** the browser DOM and JavaScript engine, the Web Storage API (key careslot_appointments), the Date object, the confirm() dialog, Bootstrap 5.3.3 and Google Fonts. **Communication:** the browser downloads Bootstrap and the fonts over HTTPS at page load; the app never sends appointment data to any server.

5. Functional Requirements

ID	Requirement
FR-1 Header, Navigation and Hero	
FR-1.1	A navbar on a teal gradient shows the brand 'CareSlot' and two links, Book and Appointments, that jump to the form and the list; links turn amber on hover
FR-1.2	The hero shows the headline 'Today's schedule, at a glance.' with a one-line description
FR-2 Statistics Dashboard	
FR-2.1	Four cards show Total, Upcoming, Completed and Cancelled, counted from all stored appointments (not affected by search or filter)
FR-2.2	Counts update instantly after any create, edit, status change or delete; Total always equals the sum of the other three
FR-2.3	Cards appear two per row on phones and four per row on medium screens and above
FR-3 Book Appointment Form	
FR-3.1	The form has every field in Section 6.1; the doctor dropdown has the four options in 6.3; the date picker blocks dates before today; the phone box accepts at most 10 characters
FR-3.2	Submitting does not reload the page; a new record gets a unique id (time in milliseconds) and status Upcoming
FR-3.3	After a successful save the record is stored, the form is cleared, the list is redrawn and the new row flashes
FR-3.4	The form is full width above the list on phones and tablets, and one third wide beside the list on large screens
FR-4 Validation and Error Display	
FR-4.1	The form uses novalidate; script.js checks on submit in this order: name, phone, doctor, date, time, past time, double booking; only the first failure is reported
FR-4.2	The error appears in a red alert box and the faulty fields get red borders; on failure nothing is saved and typed values stay; the error clears on success, reset or Cancel edit
FR-4.3	Double booking: blocked if another record (different id) has the same doctor, date and time and is not Cancelled
FR-4.4	Past booking: blocked if the chosen date and time is earlier than the current date and time
FR-5 Appointment List	
FR-5.1	A table shows Patient (name in bold, phone below), Doctor, When (formatted date and time, notes below), Status badge and Actions
FR-5.2	Rows are sorted earliest first, rebuilt from stored data after every change, and all typed text is escaped
FR-5.3	With no data the list shows 'No appointments yet. Use the form to book the first one.'; with data but no match it shows 'No appointments match your search.'
FR-5.4	On narrow screens the table scrolls sideways inside its own box; a row is lightly highlighted on hover
FR-6 Search and Filter	
FR-6.1	The search box matches part of the patient or doctor name, ignoring case, and updates on every key press
FR-6.2	The filter offers All statuses, Upcoming, Completed, Cancelled; search and filter work together; neither changes stored data
FR-7 Edit / Reschedule	
FR-7.1	Every row has an edit button: it fills the form, changes the heading to 'Edit appointment' and the button to 'Update appointment', shows 'Cancel edit' and scrolls smoothly to the form
FR-7.2	A hidden id decides update or create; saving keeps the current status and applies the same validation, but a record never clashes with itself; changing date or time reschedules it

ID	Requirement
FR-7.3	'Cancel edit' clears the form and returns to booking mode
FR-8 Status and Delete	
FR-8.1	Only Upcoming rows show Complete and Cancel buttons; they set Completed or Cancelled, and a Cancelled slot becomes free again; status cannot return to Upcoming
FR-8.2	Every row has a delete button that asks 'Delete this appointment permanently?'; OK removes the record, Cancel leaves it; deletion cannot be undone
FR-9 Data Persistence and Footer	
FR-9.1	All appointments are saved in <code>localStorage</code> under <code>careslot_appointments</code> as a JSON array after every change and loaded at start (empty if none); they survive refresh and restart on the same browser but are not shared
FR-9.2	The footer shows 'CareSlot – Appointment Management System · Data is saved in your browser'

6. Content and Field Definitions

6.1 Booking Form Fields

Field	Input type	HTML id	Compulsory	Rule
Patient name	text	name	Yes	At least 2 characters after trimming spaces
Phone number	tel	phone	Yes	10 digits, first digit 6 to 9; at most 10 characters typed
Doctor / Service	select	doctor	Yes	One of the four options in 6.3 ('Choose one...' is empty)
Date	date	date	Yes	Today or later (picker minimum)
Time	time	time	Yes	Date and time together must be in the future
Notes	textarea	notes	No	Free text, shown under the date in the list
Edit ID	hidden	editId	System	Empty when booking; holds the record id when editing

6.2 Record Structure, Doctor List and Status Values

Record: id (timestamp text), name, phone, doctor, date (YYYY-MM-DD), time (HH:MM), notes, status. The keys of the `fields` object match these names, so one loop fills the form when editing. **Doctors:** Dr. Mehta – General Physician; Dr. Khan – Dentist; Dr. Verma – Dermatologist; Dr. Singh – Pediatrician.

Status	Meaning	Badge	Buttons shown
Upcoming	Booked, not yet attended or cancelled	Light blue	Complete, Cancel, Edit, Delete
Completed	Patient was seen	Light green	Edit, Delete
Cancelled	Will not happen; slot is free again	Light red	Edit, Delete

7. Non-Functional Requirements

Area	Requirement
NFR-1 Usability	Every function is on one screen; booking needs six fields (one optional); errors show inside the form above the buttons; delete asks confirmation
NFR-2 Performance	Page loads in under 1 second on a normal laptop; the table redraws in under 200 ms for up to 500 appointments; custom CSS and JS are each under 50 KB. No claim is made about network load because data never uses the network
NFR-3 Portability	Works in Chrome, Firefox, Edge and Safari from 320 px to 1920 px; Bootstrap breakpoints plus custom rules at 576 px and 992 px; no installation or build step
NFR-4 Accessibility	Every field has a <code><label for></code> ; search and filter have <code>aria-label</code> ; error box has <code>role="alert"</code> ; text contrast at least 4.5:1; badges carry text, not colour alone; icon buttons have a <code>title</code> ; keyboard usable; reduced motion switches animation off
NFR-5 Maintainability	Colours defined once as CSS custom properties (changing the main colour is a one-line edit); purpose-based class names; commented sections; one job per function
NFR-6 Reliability	No console errors in normal use; stat cards always agree with stored data; invalid data is never saved. Known limit: damaged <code>localStorage</code> text stops the page from starting until site data is cleared
NFR-7 Security	All typed text passes through <code>escapeHTML</code> , so typed code is shown as text and never run; no data is sent to a server. There is no login, encryption or access control: data is plain text in the browser, so use sample data only

8. Design Constraints and Coding Standards

ID	Standard
STD-1	Begin with <code><!DOCTYPE html></code> , <code>lang="en"</code> , <code><meta charset="UTF-8"></code> , the viewport meta tag and a unique, descriptive <code><title></code>
STD-2	Use semantic tags: <code>nav</code> , <code>header</code> , <code>main</code> , <code>section</code> , <code>footer</code>
STD-3	No inline <code>style</code> attributes, <code><style></code> blocks or inline event handlers; CSS lives in <code>style.css</code> and JavaScript in <code>script.js</code>
STD-4	<code>style.css</code> is linked after Bootstrap's CSS; scripts sit at the end of <code><body></code> , Bootstrap bundle before <code>script.js</code>
STD-5	JavaScript uses <code>const</code> and <code>let</code> (not <code>var</code>) and strict equality (<code>===</code>); each function does one job
STD-6	Buttons created by JavaScript use one delegated listener with <code>data-action</code> and <code>data-id</code> ; typed text goes through <code>escapeHTML</code> before <code>innerHTML</code>
STD-7	Tables only for tabular data; layout with Bootstrap grid, Flexbox and CSS Grid, not floats; lowercase file names; no deprecated tags

9. Validation Rules

Two layers are used. HTML attributes stop some wrong input at the source (`maxlength="10"` on phone, `type="tel"`, `type="date"` with `min` set by JavaScript, `type="time"`, and `novalidate` on the form). Then `validate()` in `script.js` applies the rules below, top to bottom, stopping at the first failure.

ID	Check	Rule	Error message shown
VAL-1	Name	Length after trim is at least 2	Enter the patient's full name.

ID	Check	Rule	Error message shown
VAL-2	Phone	Matches /^[6-9]\d{9}\$/: first digit 6 to 9, then 9 digits	Enter a valid 10-digit mobile number.
VAL-3	Doctor	A non-empty option is selected	Choose a doctor or service.
VAL-4	Date	A date is present	Pick a date.
VAL-5	Time	A time is present	Pick a time.
VAL-6	Not in past	Date and time are not earlier than now	That time has already passed. Choose a future slot.
VAL-7	No double booking	No other non-cancelled record with same doctor, date, time (own id ignored)	[Doctor] is already booked at that time. Choose another slot.

State honestly at viva: these checks confirm the shape of data, not its truth. The clash check compares only exactly equal times (10:00 and 10:10 are both allowed). The minimum date uses the UTC date, which in India can show yesterday for a few hours after midnight. A record whose time has passed cannot be edited, because the past-time check applies to edits too.

10. Test Cases and Traceability

ID	Test	Steps	Expected result	Covers
TC01	Empty start	Clear site data, open <code>index.html</code>	Empty-list message shown; all four stats are 0	FR-2.1, FR-5.3
TC02	Empty form blocked	Click Save with every field empty	Red box 'Enter the patient's full name.'; name outlined red; nothing saved	FR-4.1, FR-4.2, VAL-1
TC03	Bad phone / missing fields	Enter 9 digits, then 5123456789; leave doctor, date, time empty in turn	Phone error; 'Choose a doctor...', 'Pick a date.', 'Pick a time.'; phone box accepts max 10 characters	VAL-2 to VAL-5, FR-3.1
TC04	Past time rejected	Pick today and a time earlier than now	'That time has already passed...' shown	FR-4.4, VAL-6
TC05	Valid booking	Fill all fields correctly; save; book an earlier slot after a later one	Row appears as Upcoming and flashes; form clears; stats update; earlier slot is listed first	FR-3.2, FR-3.3, FR-2.2, FR-5.2
TC06	Double booking	Book same doctor, date, time twice	Second is blocked with clash message	FR-4.3, VAL-7
TC07	Other doctor / cancelled slot	Same time, different doctor; then cancel one and rebook its slot	Both are saved successfully	FR-4.3, FR-8.1
TC08	Edit and reschedule	Click edit, change date, Update; edit and Update unchanged; edit then Cancel edit	Form switches mode; row moves to new date, status kept; no self-clash; Cancel edit clears form	FR-7.1 to FR-7.3
TC09	Complete / cancel	Click tick; click cross on another row	Green 'Completed'; red 'Cancelled'; buttons disappear; stats update	FR-8.1, FR-2.2
TC10	Delete	Click bin, press Cancel; click bin again, press OK	Row stays, then is removed	FR-8.2
TC11	Search and filter	Search in lower case by patient, then doctor; filter Completed; search for nothing	Only matching rows; finally 'No appointments match your search.'	FR-6.1, FR-6.2, FR-5.3
TC12	Data persists	Refresh; close and reopen browser	All appointments and statuses remain	FR-9.1
TC13	Typed code is text	Enter <code>Test</code> as name; save	Shown literally; not bold; nothing runs	FR-5.2, NFR-7
TC14	Responsive layout	View at 375, 768 and 1200 px	Stats 2 then 4 per row; form above then beside list; table scrolls inside its box	FR-2.3, FR-3.4, FR-5.4, NFR-3
TC15	Keyboard and motion	Use Tab/Enter only; enable reduced motion and add a record	All controls reachable, focus visible; row does not flash	NFR-4, UI-4

Traceability: Objectives to Requirements

Project objective	Delivered by	Project objective	Delivered by
Digitise appointment booking	FR-3, FR-5, FR-9	Overview of clinic workload	FR-2, FR-5
Reduce mistakes and double booking	FR-4, FR-7	Track outcomes, find records fast	FR-6, FR-8
Clean, responsive, usable interface	UI-1 to UI-8, NFR-1, 3, 4		

11. Future Enhancements

Not part of the current project; listed to show the direction of further work.

Phase 2: front-end only	Phase 3: backend and database
- Local date for the date-picker minimum - Safe loading of saved data with try/catch 'Overdue' flag for passed Upcoming appointments - Appointment length and slot grid to stop overlaps - Doctor working hours and leave days - Undo for status changes - Export to CSV and print the day's list - Dark mode and larger text option	- Staff and doctor login with roles - Central database shared across devices - Patient records and visit history - SMS and email reminders - Patient self-booking of free slots - Doctor dashboard - Online payment and receipts - Reports, analytics and encrypted storage

12. Conclusion

CareSlot, as specified here, is a complete single-page front-end application for booking, viewing, searching, filtering, editing, completing, cancelling and deleting clinic appointments with live statistics. It states openly what it does and does not do: data is validated (including a double-booking check) and saved in `localStorage`, but there is no login and no server. It demonstrates DOM handling, event delegation, CRUD, browser storage, safe input handling and responsive design. 15 test cases trace to requirements, and requirements to objectives. Section 11 shows how it can grow.